ClosingScript

This project may appeal to programmers who dislike the significant indentation in CoffeeScript but otherwise like the language for its simplicity and expressiveness.

ClosingScript addresses this sensitive problem by adding *closing* keywords to CoffeeScript syntax to terminate multi-line statements, making whitespace no longer significant.

ClosingScript is a text preprocessor that regenerates significant indentation using these closing keywords.

A closing keyword consists of the slash followed by the name of its matching multi-line statement.

List of closing keywords:

/**`if`**, /**`unless`**, /**`while`**, /**`until`**, /**`for`**, /**`switch`**, /**`loop`**, /**`try`**, /**`class`**, /**`end`** (function)

Advantages:

- Whitespace is not significant.
- Robust code structure.
- Clear visual termination of multi-line statements.
- Safe copy-pasting.
- Safe auto-formatting.

Disadvantages:

- Slightly more verbose syntax.

General syntax

The syntax of a multi-line statement is indentical to CoffeeScript syntax except for the addition of a closing keyword. The list below is not exhaustive and is provided to give a general idea.

```
functionName = (parameters) -> | =>
  code
/end [optional following code]                # example of following code:  /end, timeout

do ->
  code
/end

[variable =] if condition
  code
[else if condition
   code]
[else
   code]
/if

[variable =] while condition [guard clauses]
   code
/while

[variable =] for iterable [guard clauses]
   code
/for

[variable =] loop
   code
/loop

[variable =] switch [variable]
 when
    code
 [when condition then code]
 [else
    code]
/switch

class name
   code
/class

try
   code
[catch …
   code ]
[finally …
   code ]
/try
```

Usage

The preprocessor outputs to *stdout.* To get JavaScript, the official CoffeeScript compiler must be installed.

Any error message with a line number from the preprocessor or the compiler will match the same line number in a ***ClosingScript*** source.

```
To convert and compile to JavaScript in one step:
 node <path>closing.js script.coffee | coffee -s -c > script.js

To get a converted copy of a source:
 node <path>closing.js script.coffee > script.converted.coffee

To get a formatted copy of a source (only reindented consistently):
 node <path>closing.js -f script.coffee > script.formatted.coffee
```

Usage with the Geany code editor

In Linux, this command can be added to the Build menu of the Geany code editor. Its converts and compiles in one step, with errors falling back at guilty lines in the editor window, as well as error messages from the preprocessor and the compiler are sent to the ‘Compiler’ message window.
This conveniently makes the whole process as a single compiling step to the user.

set -o pipefail; **node** <path>**closing.js** "%f" | **coffee** -s -c > "%e".js 2> >(**sed** 's/\[stdin\]/%f/' >&2)